1 Network mode

This section documents the multi-station network estimator (`ProcessingMode::PPP_RTK_Network`), which *produces* the satellite-side corrections that the single-receiver user modes consume. The other modes in this reference either difference away the satellite clock and hardware biases (the differencing section) or import them as known SSR quantities (the SSR-corrections section). The network producer inverts that relationship. It estimates the satellite clock and the satellite *phase* and *code* biases as explicit states, jointly across a network of stations with known coordinates, and emits them in IGS-SSR convention for a downstream consumer. The loop is closed by design: the single-receiver PPP-RTK engine documented elsewhere is the product test, run on a station *held out* of the network solve.

Scope and status. The numerical core consists of the arrowhead normal-equation kernel (`filter/NetworkNormalEquations`) and the semantic estimator that drives it (`filter/NetworkEstimator`). It is validated on a synthetic noise-free network, where it recovers the truth up to the declared S-transform. The surrounding pipeline is complete: the `PPP_RTK_Network` enum, its forced-uncombined `DifferencingMode::None` (`measurement/MeasurementBuilder`), the explicit satellite phase-bias state in `filter/StateManager`, and the mode dispatch in `Application::run`. `Application::runNetwork` is a thin epoch-loop orchestrator that calls a fixed, ordered sequence of single-concern seams (Section 1.1). Every step derived in this chapter runs inside one of those seams: per-station design-row assembly from each station's `MeasurementModel` (`networkPassAHead_` / `networkPassB_`), the S-basis datum with its null-space rank gate (Section 1.4, in `networkSolveGlue`), the between-epoch time update, and the product emitter (`NetworkProductEmitter`, driven from `networkEmitStore`). The mathematics below describes that estimator core.

This chapter treats the single-epoch solve. The between-epoch carry is layered. The base layer is the *diagonal* D2 shared carry: each shared parameter's posterior value and variance are re-injected as a diagonal prior on the next epoch's shared border, inflated by $q\,\Delta t$ and dropped on a datum change (`networkInjectD2CarryPrior`, `app/NetworkRunContext`). This layer has no enable flag, so the producer is always a filtered solve, never a snapshot solve. The layers built on it are each a large subsystem, default off in the loader, and are referenced here without derivation: the C1 full-covariance shared carry (`network.shared_carry_full_cov`, folded through `addSharedPriorDense`), the Card-4 invariant DD carry (`network.carry_invariant`, `setDdPriors`), the station-local phase-leveling, ZWD and receiver-code-bias carries folded through `addStationPrior`, and the MW-WL, EWL and lane-coordinate widelane layers. The golden configurations (`config/golden/golden_field_refound.json`, `config/golden/golden_sim_refound.json`) arm the C1 carry and phase leveling (`network.phase_leveling`) with the float ambiguity vehicle (`network.ambiguity_vehicle` set to `float_kf`; the loader default is `level`). They leave the invariant DD carry and every widelane layer off.

1.1 Producer architecture: the decomposed epoch loop

The rest of this chapter is mathematics. This subsection maps each piece of it to the code that executes it, which is spread over several translation units.

Decomposition. The producer was a single driver function of roughly 14,700 lines. At that size it could not be reasoned about as a whole, exercised below the granularity of a full run, or reviewed: a

search for the datum build could not bound where it was *not*. It was decomposed into single-concern *seams* under the strongest available safety property. Relocation is *verbatim*: bodies move without rewriting, and each increment was gated on a matrix of golden runs required to be *byte-identical* to the pre-cut binary's output. That gate supports the claim that the equations below still describe the code. The driver shrank by roughly a factor of eight, to under 1,800 lines of epoch-loop orchestration, with no change in the numbers. Later cards have added seams to the call sequence: the file (`src/core/app/ApplicationNetwork.cpp`) exceeded 2,250 lines by 2026-08-15 and measures 3,071 lines on 2026-09-28. The placement rules of this section exist to bound that growth. The gate's coverage is incomplete. `docs/network_decomposition_plan.md` records per-increment byte gates through increment 8 (with increment 7b) only. Increment 8b has its own specification, and the later cuts (solve glue, carry install, DD-prior build, producer AR, MW/WL glue, re-level, teardown) have no gate record in that document.

Four stages. A network run has four stages: three translation units and the seam sequence between them.

- **Setup** (`app/ApplicationNetworkSetup`). `networkValidateInputs_` rejects an unrunnable configuration before anything is allocated. `networkSetup_` then builds the station roster and its synchroniser, the product sink and its declared S-basis contract, and, when the closed loop is armed, the consumer engine (Section 1.7). Its output is one `NetworkSetupResult`, filled in place and neither copyable nor movable: three run-long references pin its member addresses, so the member declaration order in that header is normative.
- **Assembly** (`app/ApplicationNetworkAssembly`), two passes per epoch. PASS A is *layout*. It runs each present station's geometry models and pre-allocates the shared satellite states, taking the datum (pivot) station *first*, so that the phase-bias columns admitted to the shared block are those backed by a pivot-receiver ambiguity pin. The local half of the datum (Section 1.4) therefore constrains the column layout as well as the solve. PASS B is *assembly*. Each station's undifferenced design block is computed with no receiver-side differencing and then *split*: the satellite clock and phase-bias columns scatter into the shared border $\boldsymbol{A}_{s,r}$, and the remainder becomes the private block $\boldsymbol{A}_{r,r}$ of Equation (1). The three receiver-position columns are dropped, because network station coordinates are held fixed. Equation (4) is formed from these two pieces.
- **The epoch seams**: the ordered sequence tabulated below, which is the body of the loop.
- **Teardown** (`filter/NetworkTeardown`). The run-end readout, the service counters and the summary writers live in `networkTeardown`, which also returns the process exit code. The driver ends with a `return` of that call.

The seam contract. A seam is a free function over the run context, the configuration, the logger and the trace bundle. It opens with an alias preamble that reconstructs the driver's former local names, so the relocated body reads as it did before the cut. Each seam header states that the seam *owns no state*: every carried object is a reference into the context. Pure relocation required two structural adaptations. A relocated block that used to `continue` the epoch loop returns an `EpochAction` of `Proceed`, `SkipEpoch` or `StopRun` (`filter/NetworkRunTypes`) instead. The per-skip bookkeeping stays verbatim inside the seam, and the caller's `continue` preserves the original bypass of the timing bracket. A block whose locals are read further down the loop returns a result struct (`NetworkSolveGlueResult` and its siblings) that the call site re-aliases.

The run context. Carried state, configuration snapshots and counters live in one object, `NetworkRunContext` (`app/NetworkRunContext`). It holds a `const NetworkKnobs` snapshot of the configuration, taken in the original getter read order, the cross-epoch `NetworkCarryState`, and `NetworkServiceCounters`. A seam introduces no function-local carry: a relocated block that leaves a local behind *forks* the carry with no diagnostic, and increment 12 deleted 191 dead aliases that this pattern had left. Persistence follows the *three-place rule*: a carry field that is a snapshot member appears in the warm-resume fingerprint, in the restore and in the snapshot write, all three in `NetworkRunContext`. A field present in only two of the three yields a warm resume that disagrees with the run that wrote it, with no error raised.

The ordered epoch seams. The table below lists the per-epoch call sequence in driver order. The driver's section timers bracket the calls. Rows marked *default off* are inert unless armed; most of the newer machinery in this producer is in that state. The table omits smaller calls in the same loop, among them the signal roster (`networkBuildSignalRoster`), the float-vehicle drop step (`networkAmbFloatApplyDrops`), the gross line-of-sight screen (`networkGrossLosScreen`, loader default off, armed by the goldens), and the code-only clock-level re-base of card CLV (`networkClockLevelRebase`, after `networkStashStatus`; inert unless `network.clock_level_carry`, default off). The gross screen runs its response half after post-fit RAIM; under `network.gross_los_emit_exclude` its decision half runs earlier, before `networkSolPrep`, so that a declaration reaches the epoch it declares. The prefit census row runs only when one of its instruments is armed: `network.trace_prefit_census` (armed by the goldens), `network.trace_topology`, or a positive `network.seed_birth_check_sigma`.

seam call (driver order)	concern (translation unit)
`networkFastClockTick`	sub-epoch fast-clock tick between solution epochs; *default off* (`filter/NetworkFastClock`)
`networkPassAHead_`	PASS A head: per-station geometry and shared-state pre-allocation, datum station first (`app/ApplicationNetworkAssembly`)
`networkPassASlipScan_`	PASS A tail and the slip scans that fill the slip funnel (`app/ApplicationNetworkAssembly`, `slip/NetworkSlipQc`)
`networkAssemblyPrefilter_`	raw slip observables; cross-station observability prune (`app/ApplicationNetworkAssembly`)
`networkSignalIntegrity`	per-LOS band-state classifier: degraded/unvetted phase demotion and regain refloat; *default off* (`filter/NetworkSignalIntegrity`)
`networkMeasurementAdmission`	product-referenced corrupt-channel screen, the one admission test a differential screen cannot see; *default off* (`filter/NetworkMeasurementAdmission`)
`networkBuildPhaseDatumCandidates`, then `networkAnchorHygiene` and `networkElectPhaseAnchors` / `networkElectEnsembleDatum` / `networkElectChainDatum`	election of the *local* half of the datum: the candidate census and coverage gate, the eq34 reattach hygiene that governs anchor-host eligibility and the fresh-pin deweight ramp, then whichever realisation is armed (`filter/NetworkPhaseDatum`, `filter/NetworkAnchorHygiene`, `filter/NetworkChainDatum`)
`networkAssemblyRegisterColumns_`	clear the layout, register this epoch's shared columns, open the arrowhead system (`app/ApplicationNetworkAssembly`)
`networkSdEwlAr`	single-difference extra-widelane integer decision; *default off* (`filter/NetworkSdEwlAr`)
`networkRxCbDemeanPrepass`	receiver code-bias carry de-mean before assembly reads the map; *default off* (`app/NetworkRunContext`)
`networkPassB_`	PASS B produce/consume: the split design blocks handed to `addStation` (`app/ApplicationNetworkAssembly`, `filter/NetworkCoupling`)
`networkBuildEnsembleGauges`, `networkBuildCarryDdPriors`	sum-to-zero phase gauges; Card-4 DD carry priors (`filter/NetworkDdPriorBuild`)
`networkSolveGlue`	shared datum build, rank gate and drop-and-heal, D2 carry injection, the constrained solve, within-epoch RAIM re-solve, withhold gates (`filter/NetworkSolveGlue`, calling `NetworkSbasis`, `NetworkPhaseDatum`, `NetworkRelevel`)
`networkInstallCarry`	DD posterior read-back into the invariant carry; *default off* (`filter/NetworkInstallCarry`)
`networkProducerAr`	producer-side AR diagnostic and normal-equation export; read-only and log-only, *default off* (`filter/NetworkProducerAr`)
`networkSolPrep`	map to the solution epoch; per-satellite discontinuity signalling (`filter/NetworkSolPrep`)
`networkEmitStore`	emit re-base, phase-bias continuity filter, store push, product write, closed-loop consume (`filter/NetworkEmitStore`)
`networkFastClockHarvest`	fast-clock harvest off the accepted solution; *default off* (`filter/NetworkFastClock`)
`networkIonoDiag`, `networkDdFrac`, `networkShDevVtec`, `networkShHealth`, `networkIonoDwRelease`, `networkLeveling`	diagnostic sweeps, the iono-deweight release hygiene that reseeds slant states a zero-weight era left unvalidated, and the SH-deviation and phase-leveling carry harvests (`filter/NetworkIonoDiag`, `NetworkDdFrac`, `NetworkShDevVtec`, `NetworkShHealth`, `NetworkIonoDwRelease`, `NetworkLeveling`)
`networkPostFitRaim`	post-fit network RAIM and its attribution pass (`filter/NetworkRaim`)
`networkClockQc`	two-state clock update and clock fault QC (`filter/NetworkClockQc`)
`networkPostfitSigma0`, `networkShSpectrum`	post-fit variance factor and reduced-normal spectrum; *default off* (`filter/NetworkFrameDiag`)
`networkPrefitCensus`	the only always-on *pre*-solve screen: prefit innovation census, structural fuse, seed-birth tripwire and estimation-topology ledger (`filter/NetworkPrefitCensus`)
`networkRedundancyCensus`	row redundancy and testability, the statistical parked-garbage monitor; measure-only, *default off* (`filter/NetworkRedundancyCensus`)
`networkBlockBaardaEmit`	(station, satellite) LOS block outlier census; measure-only, *default off* (`filter/NetworkBlockBaarda`)
`networkCongruenceEmit`	station-congruence screen: coordinate-fault detection in position space; measure-only, *default off* (`filter/NetworkCongruence`)
`networkMdbFlush`	Baarda reliability numbers and minimal detectable bias on the

Reading key. The three derivations that follow map to three places in that sequence. The rank deficiency and its minimum-constraint datum (Section 1.4) are built and gated inside `networkSolveGlue`. The arrowhead Schur reduction (Section 1.5) runs in the kernel that the same seam invokes, on normals accumulated station by station during PASS B. The closed loop (Section 1.7), meaning emission, product write and, when armed, the held-out consumer step, runs in `networkEmitStore`. The rest of the table is quality control, diagnostics and carry bookkeeping around those three.

1.2 Joint multi-station observation model

The producer uses the same *undifferenced, uncombined* observations as single-receiver PPP (the observation-equations section), but stacks the equations of every station $r \in \mathcal{R}$ into one system. The unknowns partition into a block *shared* by all stations and a block *private* to each:

- **Shared (satellite-side).** The estimable satellite clock $c\,\mathrm{d}t^i$, the per-signal satellite code bias B^i_s, and the per-signal satellite *phase* bias $\bar{b}^i_s$. These columns are identical for every station, since a satellite's clock and hardware delays do not depend on the observer. Orbits are held fixed to a precise product; orbit determination is outside the producer's scope. When `network.estimate_iono_sh` is set, the shared block carries a fourth family, network-wide rather than per-satellite: the coefficients a_{nm}, b_{nm} of a global single-layer spherical-harmonic VTEC expansion (`SharedParamKind::VtecSh`, registered through `NetworkEstimator::registerVtecCoeff`). An optional second, plasmasphere-side shell is set by `estimation.global_vtec_sh.layer2_max_degree` (default 0, i.e. single-layer); the coefficients of both shells coexist on the shared border, and the pin row sums them. This co-estimated ionosphere model replaces the external GIM. It also forces satellite code-bias estimation on, since the two are jointly required for the per-satellite slant-iono columns below, unless `network.external_sat_code_bias` takes the satellite code biases from the external `.BIA`. Its columns Schur-reduce onto the shared border in the same way as the satellite-indexed families. The goldens leave `network.estimate_iono_sh` off and estimate a free per-station, per-satellite slant ionosphere instead.
- **Private (station-side).** For each station r: its receiver clock $c\,\mathrm{d}t_r$, the inter-system time offsets $c\,\delta t^{(g)}_r$, the zenith wet troposphere ZWD_r (and gradients), the per-satellite slant ionosphere $I^i_{r,1}$, the receiver biases, and one float ambiguity $N^i_{r,s}$ per tracked carrier. Under the chain datum only (`network.phase_datum_eq34`, default off), the block also holds a per-(receiver, system, signal) receiver *phase*-bias split $\delta_{r,s}$, re-derived each epoch (`StateKeys::makeReceiverPhaseBiasKey`). Under that realisation it keeps the receiver's fractional phase bias out of the *emitted* satellite phase bias. Station coordinates are known and held fixed, so position is not a free parameter.

Write the shared satellite-parameter vector as $\boldsymbol{x}_s$ and station r's private nuisance vector as $\boldsymbol{x}_r$. The undifferenced observation rows of station r linearise (the observation-equations section) to

$$\boldsymbol{y}_r \;=\; \boldsymbol{A}_{s,r}\,\boldsymbol{x}_s \;+\; \boldsymbol{A}_{r,r}\,\boldsymbol{x}_r \;+\; \boldsymbol{\varepsilon}_r, \qquad r \in \mathcal{R}, \tag{1}$$

where $\boldsymbol{y}_r$ is the prefit residual (observed minus computed), $\boldsymbol{A}_{s,r}$ is the Jacobian with respect to the *shared* satellite columns, and $\boldsymbol{A}_{r,r}$ is the Jacobian with respect to station r's *own* nuisances. There is no $\boldsymbol{A}_{r,r'}$ term for $r' \neq r$: a station's observations touch only the shared block and its own private block. The reduction of Section 1.5 rests on that sparsity.

No receiver-side differencing. Unlike the PPP-RTK user mode, the producer applies *no* between-satellite or between-receiver differencing. The differencing strategy is forced to `DifferencingMode::None` (`MeasurementBuilder.cpp`, `differencingModeFor`), overriding any `between_satellite_sd`

toggle in a network config. Differencing would cancel the satellite-side terms the producer *estimates*. The network carries them as explicit columns instead and resolves the resulting rank deficiency with a minimum-constraint datum (Section 1.4).

1.3 The explicit satellite phase-bias state

The main contrast with the user side is the treatment of the satellite phase bias b_s^i (written $\bar{b}_s^i$ once promoted to a state). On the user side this term is never an explicit unknown: between-satellite single differencing (the differencing section) cancels the *receiver* phase bias, the satellite phase bias arrives as a *known* SSR product, and what remains is an integer single-difference ambiguity. The user handles biases *implicitly*.

The producer *generates* that known product, so it carries the satellite phase bias as an estimated state. The state vector gains a per-(satellite, signal) phase bias column,

$$\bar{b}_s^i \in \boldsymbol{x}_s, \qquad \text{keyed by } \texttt{makeSatPhaseBiasKey}(i, s) = \texttt{"}i\texttt{|}s\texttt{|PB"}, \tag{2}$$

held in `StateManager::sat_phase_bias_indices_`, a map kept disjoint from the satellite *code* bias map (`sat_bias_indices_`); the trailing `|PB` discriminator keeps the two key namespaces apart (`include/filter/StateKeySchema.hpp`). Its process noise is a small random walk (`network.process_noise.sat_phase_bias`, legacy `filter.process_noise.sat_phase_bias_q`; default $1 \times 10^{-12}\,\mathrm{m^2\,s^{-1}}$, two orders of magnitude below the `sat_code_bias_q` default of $1 \times 10^{-10}\,\mathrm{m^2\,s^{-1}}$). Satellite phase hardware delays are nearly constant but not strictly so: a $q = 0$ random walk would over-converge the diagonal of $\boldsymbol{P}$ and leave the column insensitive to real slow drift (`src/core/filter/StateManager.cpp`).

Remark 1. The estimated $\bar{b}_s^i$ is not the raw satellite phase hardware delay, which is not separately estimable. It is the delay *lumped with the network pivot-receiver ambiguity/clock datum* (Odijk's δ^s, [1]): the estimable combination that, when removed by a user receiver, restores the integer property of that user's single-difference ambiguities. Because of this lumping the phase-bias product must be *consistent* (the SSR-corrections section): the producer's datum and the consumer's integer recovery must use the same S-system.

1.4 Rank deficiency and the minimum-constraint S-basis

The undifferenced network system Equation (1) is rank deficient. The linear dependencies that the user side removes by differencing appear here as exact *gauge freedoms* of the joint normal matrix. No observation distinguishes a common shift of all satellite clocks from an opposite shift of all receiver clocks; none fixes the split of an absolute phase bias between the satellite column and the per-station ambiguity datum; none fixes an inter-system bias (ISB) common to a constellation. These are the S-system ambiguities of [2]: the data fix only *estimable combinations*, and a *datum* must be chosen to pin the remaining null space.

Exact gauge, soft elimination. The datum is a *minimum-constraint* datum applied as exact hard pins to the reduced shared system, never a structural elimination of columns. Each gauge is realised as a single *Dirichlet pin* of one shared column c to a datum value (almost always 0): the column's known contribution moves to the right-hand side, its row and column in the reduced normal matrix are zeroed, and a diagonal entry is set so that $x_c =$ value holds exactly (`NetworkEstimator::solve`, `src/core/filter/NetworkEstimator.cpp`). The pin is applied to a *copy* of the reduced normals (the kernel's `solveSharedConstrained` seam). The estimator's own reduced system stays

unmodified, so per-station back-substitution (Section 1.5) stays valid. The pinned diagonal is set to the reduced normals' own scale (the largest diagonal magnitude) rather than to unity. An *under*-specified datum then still appears as a near-zero *relative* pivot in the positive-definiteness guard, independent of the network's overall weighting magnitude.

In S-transform language, let $\boldsymbol{x}$ collect the rank-deficient undifferenced parameters and $\boldsymbol{S}$ be the S-basis selection. The emitted satellite corrections are the estimable combination $\boldsymbol{S}\,\boldsymbol{x}$. Choosing the datum is choosing $\boldsymbol{S}$, and the hard pins above are the minimal set of rows of $\boldsymbol{S}$ that remove the deficiency. The two halves of the datum live in two places:

- **Shared-block gauges** touch the shared satellite columns: the overall clock datum (one per constellation), the per-constellation phase-bias↔ambiguity datum, and the ISB datum. They go through the explicit `NetworkEstimator::Datum` (a list of column pins) and are applied in `solve`.
- **Local datum elements** live on one station's private block: a network *pivot receiver*'s own ambiguity/clock pin, which the emitted satellite phase bias absorbs (Odijk Eq. 4/9). The *caller* realises them as tight prior rows folded into that station's $\boldsymbol{A}_{r,r}$, $\boldsymbol{W}$, and $\boldsymbol{y}$, not through the shared `Datum` (`NetworkEstimator.hpp`). Station priors also make each local block $\boldsymbol{N}_{rr}$ full rank for the Schur step (Section 1.5).

Selectable datum realisations. The pins just described are the loader defaults. Each half of the datum admits alternatives, selected by config and recorded in the `SbasisDeclaration`. All produce the same estimable $\boldsymbol{S}\,\boldsymbol{x}$ and differ in how they distribute the gauge. The clock/ISB datum has three modes (`SbasisDeclaration::IsbDatum`): the hard `PerConstellationSatClockPin` above (default); a `SumToZeroPerConstellation` constraint (`network.clock_datum_sum_to_zero`) that spreads the pin across all satellites of a constellation; and a `BroadcastAnchoredPerSatellite` soft anchor (`network.clock_datum_broadcast_anchored`) that ties each satellite clock to its a-priori value (the broadcast clock under `network.apriori_source brdc_rts`, the precise clock under `sp3`) at a declared 1σ. The ambiguity↔phase-bias datum also has three modes (`AmbDatum`). The first is the local `PivotReceiverZeroAmbiguity` pin above (default). Its anchor may be elected *per* (satellite, signal) at the highest-coverage tracking receiver instead of at the single clock pivot (`network.phase_datum_prioritized`, default off; `networkElectPhaseAnchors`, `filter/NetworkPhaseDatum`). This is the same one-pin-per-freedom constraint realised at a different station, and it keeps a clean gauge for a satellite that is low or unseen at the clock pivot. The other two modes are a `SumToZeroPerReceiver` ensemble (`network.phase_datum_ensemble`) and a `SingleReceiverChain` realisation (`network.phase_datum_eq34`). When satellite code biases are estimated (the W6 slice, `network.estimate_sat_code_bias`), their gauge, a per-constellation sum-to-zero (`CodeBiasDatum::SumToZeroPerConstellation`), is folded as explicit constraint rows through `NetworkEstimator::addSharedConstraintRows` rather than as column pins. With no ionosphere constraint, `network.sat_gf_datum` (`none`, `zero` or `broadcast_tgd`; default `none`) adds per-satellite geometry-free code-bias datum rows. Whichever realisations are active, the rank gate below is declared against the matching S-basis dimension. The goldens keep the default clock pin (at `network.clock_datum_station`), arm the prioritized phase datum with `network.phase_datum_held_basis`, estimate the satellite code biases, and pin the geometry-free code bias at the broadcast $\mathrm{T_{GD}}$ (`broadcast_tgd`).

Remark 2. A null-space rank gate eigen-decomposes the pre-constraint reduced normals and asserts that the count of small eigenvalues equals the *declared* S-basis dimension. It is implemented as `NetworkSbasis::assertNullSpaceRank` and is invoked on `reducedSharedNormals()` *before* the constrained solve, against the `SbasisDeclaration`'s `declared_dimension`. The call sits in the

solve seam `networkSolveGlue` (`filter/NetworkSolveGlue`), which also builds the datum through `NetworkSbasis::buildSharedDatum` (Section 1.1). The gate complements the positive-definiteness guard. The guard catches the exact-deficiency (under-specified datum) case on the pinned pivot; the rank gate checks the reduced deficiency against the declared gauge dimension and withholds any epoch whose null-space count disagrees. Under the prioritized phase datum the gate first attempts a *drop-and-heal* pass that soft-anchors the near-null phase-bias columns (`healNearNullPhaseBias`, `filter/NetworkPhaseDatum`) and then re-gates once. On the legacy pivot datum there is nothing to heal: the heal returns zero and the epoch is withheld immediately.

1.5 Per-station Schur reduction (arrowhead normals)

Stacking Equation (1) over all stations and forming weighted normal equations $\boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{A}\,\boldsymbol{x} = \boldsymbol{A}^{\mathrm{T}}\boldsymbol{W}\boldsymbol{y}$ yields a *bordered block-arrow* ("arrowhead") system. The station–station off-diagonal blocks are exactly zero, because stations couple only through $\boldsymbol{x}_s$:

$$\begin{bmatrix} \boldsymbol{N}_{ss} & \boldsymbol{N}_{s1} & \boldsymbol{N}_{s2} & \cdots & \boldsymbol{N}_{sR} \\ \boldsymbol{N}_{1s} & \boldsymbol{N}_{11} & \boldsymbol{0} & \cdots & \boldsymbol{0} \\ \boldsymbol{N}_{2s} & \boldsymbol{0} & \boldsymbol{N}_{22} & \cdots & \boldsymbol{0} \\ \vdots & & & \ddots & \\ \boldsymbol{N}_{Rs} & \boldsymbol{0} & \boldsymbol{0} & \cdots & \boldsymbol{N}_{RR} \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_s \\ \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \\ \vdots \\ \boldsymbol{x}_R \end{bmatrix} = \begin{bmatrix} \boldsymbol{u}_s \\ \boldsymbol{u}_1 \\ \boldsymbol{u}_2 \\ \vdots \\ \boldsymbol{u}_R \end{bmatrix}, \tag{3}$$

with the per-station blocks built from that station's pre-formed design rows,

$$\boldsymbol{N}_{ss}^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{A}_{s,r}, \quad \boldsymbol{N}_{sr}^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{A}_{r,r}, \quad \boldsymbol{N}_{rr}^{r} = \boldsymbol{A}_{r,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{A}_{r,r}, \tag{4}$$

and $\boldsymbol{u}_s^{r} = \boldsymbol{A}_{s,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{y}_r$, $\boldsymbol{u}_r^{r} = \boldsymbol{A}_{r,r}^{\mathrm{T}}\boldsymbol{W}_r\boldsymbol{y}_r$. A full (non-diagonal) $\boldsymbol{W}_r$ is supported, so correlated undifferenced measurement covariance (e.g. shared pre-corrections) propagates exactly (`addStationObservations`, `NetworkNormalEquations.hpp`).

Lossless marginalisation. The arrowhead system is solved by Schur-eliminating each station's private block onto the shared border. Each $\boldsymbol{N}_{rr}^{r}$ is factorised once ($\mathrm{LDL}^{\mathrm{T}}$) when the station is added, and `reduce` and `reconstructStation` reuse the factor. The elimination accumulates

$$\tilde{\boldsymbol{N}}_{ss} = \sum_{r\in\mathcal{R}} \left(\boldsymbol{N}_{ss}^{r} - \boldsymbol{N}_{sr}^{r}\,(\boldsymbol{N}_{rr}^{r})^{-1}\,\boldsymbol{N}_{sr}^{r\mathrm{T}} \right) \;+\; \boldsymbol{N}_{ss}^{\mathrm{prior}}, \tag{5}$$

$$\tilde{\boldsymbol{u}}_s = \sum_{r\in\mathcal{R}} \left(\boldsymbol{u}_s^{r} - \boldsymbol{N}_{sr}^{r}\,(\boldsymbol{N}_{rr}^{r})^{-1}\,\boldsymbol{u}_r^{r} \right) \;+\; \boldsymbol{u}_s^{\mathrm{prior}}, \tag{6}$$

the reduced shared system $\tilde{\boldsymbol{N}}_{ss}\,\boldsymbol{x}_s = \tilde{\boldsymbol{u}}_s$ (`NetworkNormalEquations::reduce`). The reduction is a *lossless* marginalisation. Solving Equations (5) and (6) for $\boldsymbol{x}_s$ and back-substituting per station,

$$\boldsymbol{x}_r = (\boldsymbol{N}_{rr}^{r})^{-1}\big(\boldsymbol{u}_r^{r} - \boldsymbol{N}_{sr}^{r\mathrm{T}}\,\boldsymbol{x}_s\big), \tag{7}$$

reproduces the dense stacked solve to machine precision. Fill-in is confined to the satellite border and each station's elimination is independent, so the cost is linear in the station count and the per-station eliminations run in parallel (`reconstructStation`, `NetworkNormalEquations.cpp`). The shared-block priors $\boldsymbol{N}_{ss}^{\mathrm{prior}}$ in Equation (5) (soft physical priors and any additive exact-gauge contributions) are added before the reduction through `addSharedInfo` and pass through the Schur complement unchanged.

Remark 3. This arrowhead topology with exact reduction mirrors the NGS M-PAGES `NgsLsq::NormalEquations::reduce` / `reconstructLocals` batch network solver. In the specialisation, the M-PAGES "global" block is the shared satellite border and the per-epoch swept block becomes the R independent station blocks. A two-stage covariance-form alternative was rejected because the normal-equation Schur form is exact and cheaper here.

1.6 Constellation scope

The producer supports GPS, Galileo and BeiDou on the same footing, selected by `processing.active_systems`. The goldens are GPS-only (L1/L2). The Galileo extension of the golden posture (card GAL, `docs/refound/GAL_gps_galileo_dual_2026-09-22.md`) is built, default off and not armed. GLONASS phase biases are excluded. The reason is structural: GLONASS is FDMA, so each satellite transmits on a slightly different frequency, and the receiver inter-channel (hardware) bias does *not* cancel across satellites as it does for the CDMA constellations. The satellite/receiver phase-bias bookkeeping that underpins the explicit-bias S-system (Sections 1.3 and 1.4) assumes a per-signal bias common to a constellation's satellites. The per-frequency GLONASS inter-channel bias breaks that assumption and would corrupt the phase datum. GLONASS is therefore excluded from the network *phase*-bias product by design (`docs/ssystem_conformance_2026-06-15.md`, now at `docs/attic/ssystem_conformance_2026-06-15.md`); the user side excludes it from ambiguity resolution for the same reason. A *code-only* GLONASS mode is available behind `network.estimate_glonass` (`enableGlonassNetwork`, default off). It admits GLONASS to the solve with a satellite clock and a per-slot receiver inter-frequency bias but *no* FDMA phase bias, so the phase-datum bookkeeping above is unaffected. The mode is mutually exclusive with the W6 satellite code-bias slice: GLONASS admission skips the GPS/Galileo receiver DCB that code-bias separation needs, so `network.estimate_sat_code_bias` together with `network.estimate_glonass` is a hard load error. The same exclusion applies to the co-estimated SH-VTEC ionosphere, which forces the code-bias slice on (unless `network.external_sat_code_bias` is set), and to the goldens, which estimate satellite code biases. This mode is distinct from the float-only GLONASS support that the user engine carries for positioning. With the code-only mode off, the network *producer* generates no GLONASS bias products. Broadcast satellite health enters the network path through `network.sat_health` (card HLT; `off`, `pivot` or `exclude`, default `off`; built, not armed). Under `exclude`, setup removes a satellite's observations from the synchronised epochs at which the broadcast flags it; under `pivot`, only the clock-pivot election reads the health series.

1.7 The closed loop: producer → held-out user

The emitted product is one `NetworkSolutionEpoch` per content epoch: per-satellite clock C_0 plus per-signal code and phase biases, keyed by RINEX satellite id and serialised in IGS-SSR convention by `NetworkProductEmitter` (`NetworkEstimator::toSolutionEpoch`). That product is the SSR stream the single-receiver engine already consumes: the satellite clock refines $-c\,\mathrm{d}t^i$, and the observable-specific biases refine $-B^i_s$ and $-b^i_s$ as *known* corrections, restoring the integer ambiguity property as described in the SSR-corrections section.

The validation strategy follows from this symmetry. The companion `ProcessingMode::PPP_RTK_User` mode runs the ordinary uncombined PPP-RTK pipeline but consumes the network's own bias/atmosphere product. A station held *out* of the network solve and positioned as a user against the produced corrections is a closed-loop product test: its recovered position and fixed ambiguities show whether the producer's S-system datum and the consumer's integer recovery are mutually consistent. The `PPP_RTK_User` consumer runs through the standard PPP path and is not a separate

estimator; only the producer takes the joint multi-station pipeline of this section.

The test has two mechanised forms. The *in-process* harness (`network.closed_loop.enable`) builds a consumer engine from a derived configuration during setup and gives it an *owned* SSR correction store instead of the process-wide one, so the producer's products cannot leak into a store that another component reads. The emit seam `networkEmitStore` steps the consumer after that epoch's products have been pushed, which closes the loop *within* the epoch. The consumer runs a between-satellite single-difference user model by default, or an undifferenced one under `closed_loop.consumer_undifferenced`. The goldens arm the harness with the undifferenced consumer on a held-out station (WTZZ00DEU on the field bed, SZZ000SIM on the sim bed). The *offline replay* driver (`network.closed_loop.replay_products_path` → `Application::runConsumerReplay`, `app/ApplicationConsumerReplay`) short-circuits `runNetwork` before input validation and producer setup. It fills the same owned store from an already-written product file through the same reader path, so the recorded stream's discontinuity bookkeeping is reconstructed exactly, and a position-domain arm costs minutes instead of a producer run. One limit applies to what a replay arm proves: the in-process SH-VTEC slant-ionosphere supply has no product-wire record, so `closed_loop.consume_network_iono` is inert on the replay path (the driver logs a warning), and the consumer floats its own ionosphere there.

References

[1] D. Odijk. "Fast precise GPS positioning in the presence of ionospheric delays". In: *Publications on Geodesy* 52 (2003).

[2] P. J. G. Teunissen. "Generalized inverses, adjustment, the datum problem and S-transformations". In: *Optimization and Design of Geodetic Networks* (1985), pp. 11–55.